PLATEFORME DE TRAITEMENT DES PAIEMENTS

REQUETES SQL & NOSQL

Nature du document	
Version	**1.0**
Client	**Étude de cas – STRIPE**
Date de la version	**08/10/2026**
Description de la version	**Bloc 2 - certification RNCP41993**
Modules concernés	**Livrable 8**
Auteur	**Ronan GUILLOU**

Table des matières

1 Stratégie de relation.....Erreur ! Signet non défini.

1.1 Définitions..... **Erreur ! Signet non défini.**

2 Collections.....Erreur ! Signet non défini.

2.1 Définitions..... **Erreur ! Signet non défini.**

2.2 sessions..... **Erreur ! Signet non défini.**

2.3 click_events..... **Erreur ! Signet non défini.**

2.4 logs..... **Erreur ! Signet non défini.**

2.5 customer_feedback..... **Erreur ! Signet non défini.**

2.6 ml_features..... **Erreur ! Signet non défini.**

2.7 inbound_payloads..... **Erreur ! Signet non défini.**

3 Compteurs Redis.....Erreur ! Signet non défini.

3.1 Contexte..... **Erreur ! Signet non défini.**

3.2 Compteurs retenus..... **Erreur ! Signet non défini.**

Note sur l'usage de l'intelligence artificielle

Pour rédiger ce document, j'ai utilisé un assistant d'IA générative comme outil d'aide à la rédaction. Il est intervenu sur quatre points : la correction de l'orthographe et de la syntaxe, la proposition de structures et de brouillons de texte que j'ai relus et modifiés, la comparaison d'idées et d'options avant décision, et la vérification de la cohérence d'ensemble.

Les choix retenus, la sélection des besoins et la validation finale de chaque chapitre relèvent de ma responsabilité.

1 Requêtes SQL sur OLTP

1.1 Paiements et fraude

Question *: Quels paiements ont été bloqués par le modèle sur les dernières 24 heures ?*

```sql
SELECT t.transaction_id,
      t.created_at,
      t.merchant_id,
      t.customer_id,
      t.amount,
      fa.score,
      fa.decision,
      fa.reasons,
      fa.model_version
FROM
      payment.transaction AS t
      JOIN payment.fraud_assessment fa
              ON fa.transaction_id = t.transaction_id
              AND fa.transaction_created_at = t.created_at
WHERE
      t.created_at >= now() - interval '24 hours'  // PostgreSQL
      AND fa.decision = 'block'
ORDER BY
      fa.score DESC;
```

Question *: Cette adresse IP cliente est-elle bloquée pour un marchand (liste interne)?*

```sql
SELECT EXISTS (
  SELECT 1
  FROM
        actor.ip_blocklist
  WHERE
      ip_range >>= '203.0.113.7'::inet
      AND (expires_at IS NULL OR expires_at > now())
      AND (merchant_id IS NULL OR merchant_id = 17)
) AS ip_blocked;
```

Question *: Une même carte (empreinte) est-elle utilisée par plusieurs clients ?*

```sql
SELECT
      fingerprint,
      COUNT(DISTINCT customer_id) AS nb_customers
FROM
      payment.payment_method
WHERE
      fingerprint IS NOT NULL
GROUP BY
      fingerprint
HAVING
      COUNT(DISTINCT customer_id) > 1
ORDER BY
      nb_customers DESC;
```

Question *: Quels clients dépassent 3 paiements en une heure (test de cartes possible) ?*

```sql
SELECT customer_id,
       COUNT(*) AS nb_payments,
       COUNT(DISTINCT payment_method_id) AS nb_cards,
       COUNT(DISTINCT device_id) AS nb_devices,
       ROUND(SUM(amount * change_rate), 0) AS total_eur
FROM
      payment.transaction
WHERE
      transaction_type = 'payment'
      AND created_at >= now() - interval '1 hour'
GROUP BY
      customer_id
HAVING
      COUNT(*) > 3
ORDER BY
      nb_payments DESC;
```

1.2 Demande de suppression

Question : *Quelles demandes d'effacement sont en retard sur leur échéance ?*

```sql
SELECT
      data_subject_request_id,
      customer_id,
      request_type,
      status,
      requested_at,
      due_date,
      current_date - due_date AS days_late
FROM
      actor.data_subject_request
WHERE
      status NOT IN ('completed', 'rejected')
      AND due_date < current_date
ORDER BY
      due_date;
```

Question : *Comment prouver qui a modifié quoi pour ce client ?*

```sql
SELECT
      table_name,
      record_id,
      action,
      changed_columns,
      changed_by,
      created_at
FROM
      audit.audit_log
WHERE
      (table_name = 'actor.customer'
      AND record_id = '1204')
              OR table_name = 'payment.payment_method'
ORDER BY
     created_at;
```

2 Requêtes SQL sur OLAP

2.1 Analyse du CA

Question : *Quel est le CA net par mois et pays du marchand ?*

```sql
SELECT
      d.year_month,
      m.country_code AS merchant_country,
      ROUND(SUM(f.signed_amount_eur), 2) AS net_revenue_eur,
      SUM(CASE
            WHEN f.transaction_type = 'payment' AND f.status = 'approved'
            THEN 1
            ELSE 0
      END) AS nb_payments
FROM
      gold.fact_transaction f
      JOIN gold.dim_date d
           ON f.date_key = d.date_key
      JOIN gold.dim_merchant m
           ON f.merchant_key = m.merchant_key
GROUP BY
      d.year_month,
      m.country_code
ORDER BY
     d.year_month,
     net_revenue_eur DESC;
```

Question : *Quels sont les 3 marchands qui rapportent le plus chaque mois ?*

```sql
WITH ranked AS (
  SELECT
      d.year_month,
      m.name AS merchant_name,
      SUM(f.signed_amount_eur) AS net_eur,
      RANK() OVER (PARTITION BY d.year_month ORDER BY SUM(f.signed_amount_eur) DESC) AS rank
  FROM
      gold.fact_transaction f
      JOIN gold.dim_date d
           ON f.date_key = d.date_key
      JOIN gold.dim_merchant m
           ON f.merchant_key = m.merchant_key
  GROUP BY
      d.year_month, m.name
)

SELECT * FROM ranked WHERE rank <= 3 ORDER BY year_month, rank;
```

Question : *Quels produits et catégories rapportent le plus, limité à 10 ?*

```sql
SELECT
      c.name AS category,
      p.name AS product,
      ROUND(SUM(i.signed_line_amount_eur), 2) AS net_eur
FROM
      gold.fact_payment_item i
      JOIN gold.dim_product p
           ON i.product_key = p.product_key
      JOIN gold.dim_analytic_category c
           ON i.analytic_category_key = c.analytic_category_key
GROUP BY
      c.name,
      p.name
ORDER BY
      net_eur DESC
LIMIT 10;
```

2.2 Segmentation client

Question *: Segmentation client par quartiles de valeur ?*

```sql
WITH net_by_customer AS (
  SELECT
      f.merchant_key,
      f.customer_key,
      SUM(CASE
           WHEN f.transaction_type = 'payment' AND f.status = 'approved'
           THEN 1
           ELSE 0
      END) AS nb_payments,
      SUM(f.signed_amount_eur) AS net_eur
  FROM
      gold.fact_transaction f
  GROUP BY
      f.merchant_key,
      f.customer_key
)
SELECT
      merchant_key,
      customer_key,
      nb_payments,
      ROUND(net_eur, 2) AS net_eur,
      NTILE(4) OVER (PARTITION BY merchant_key ORDER BY net_eur) AS quartile
FROM
      net_by_customer
ORDER BY
       merchant_key, value_quartile DESC;
```

2.3 Qualité des données

Question *: Le net par produit est-il cohérent avec le net par transaction ?*

```sql
WITH
net_transaction AS (
  SELECT
      d.year_month,
      SUM(f.signed_amount_eur) AS net_trans
  FROM ²
      gold.fact_transaction f
      JOIN gold.dim_date d
           ON f.date_key = d.date_key
  GROUP BY
       d.year_month
),

Net_product AS (
  SELECT
      d.year_month,
      SUM(x.signed_line_amount_eur) AS net_items
  FROM
      gold.fact_payment_item x
      JOIN gold.dim_date d
           ON x.date_key = d.date_key
  GROUP BY
       d.year_month
)

SELECT
      t.year_month,
        ROUND(t.net_trans, 2) AS net_transactions,
        ROUND(i.net_items, 2) AS net_items,
        ROUND(t.net_trans - i.net_items, 2) AS gap_eur
FROM
      net_trans t
      JOIN net_product i
           ON t.year_month = i.year_month
ORDER BY
     t.year_month;
```

3 Requêtes sur NoSQL

3.1 Fraude comportementale

Question *: Quelles sessions de la dernière heure ressemblent à un bot ?*
Croise les signaux bruts de comportement côté client avec la session.

```
db.ml_features.aggregate([
  // garde les signaux collectés depuis moins d'une heure, où du texte a été collé ou où la saisie dépasse 600 caractères par minute.
  { $match: {
      collected_at: { $gte: new Date(Date.now() - 60 * 60 * 1000) },
      $or: [
        { "signals.paste_detected": true },
        { "signals.typing_speed_cpm": { $gt: 600 } }
      ]
  }},
  // cherche la session correspondante dans sessions grâce à session_id avec lookup
  { $lookup: { from: "sessions", localField: "session_id",
               foreignField: "session_id", as: "session" } },
  // aplatit le résultat, un seul objet session par ligne
  { $unwind: "$session" },
  // garde que les champs utiles
  { $project: {
      _id: 0, session_id: 1, customer_id: 1,
      typing_speed_cpm: "$signals.typing_speed_cpm",
      paste_detected: "$signals.paste_detected",
      device_type: "$session.device.type",
      ip_country_id: "$session.ip_country_id",
      nb_clics: "$session.stats.nb_clics",
      duration_s: "$session.stats.duration_s"
  }},
  // met en tête les saisies les plus rapides.
  { $sort: { typing_speed_cpm: -1 } }
])
```

3.2 Retours clients et incidents

Question *: Quelles erreurs techniques provoquent les avis négatifs ?*

```
db.customer_feedback.aggregate([
   // garde les avis notés 2 ou moins sur les 7 derniers jours.
   { $match: {
      "content.rating": { $lte: 2 },
      created_at: { $gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) }
  }},
  // idem précédemment, lookup de table
  { $lookup: { from: "logs", localField: "transaction_id",
               foreignField: "transaction_id", as: "log" } },
  { $unwind: "$log" },
  // que les logs de niveau error
  { $match: { "log.level": "ERROR" } },
  //groupe par marchand, service et code HTTP, puis compte les avis, calcule la note moyenne et la durée moyenne de l'appel, et garde un exemple de texte avec first
  { $group: {
      _id: { merchant_id: "$merchant_id",
             service: "$log.context.service",
             http_status: "$log.context.http_status" },
      nb_feedbacks: { $sum: 1 },
      avg_rating: { $avg: "$content.rating" },
      avg_duration_ms: { $avg: "$log.context.duration_ms" },
      example_text: { $first: "$content.text" }
  }},
  { $sort: { nb_feedbacks: -1 } }
])
```

Note : les index ne sont pas optimisés